

International Islamic University Islamabad

Faculty of Engineering and Technology

Department of Electrical and Computer Engineering



AI & Machine Learning Lab

Experiment No. 6: Design and Implementation of AI Search Algorithms for Pathfinding and Game Decision Making

Name of Student: Bushra Nazir

Registration No.: 1071-F22

Date of Experiment: 10-3-2026

Submitted To: Engr.Asad

Design and Implementation of AI Search Algorithms for Pathfinding and Game Decision Making

Objectives

- To implement fundamental AI search algorithms Breadth-First Search (BFS), Depth-First Search (DFS), A* Search, and Minimax using Python.
- To construct and execute algorithmic solutions for practical problems such as pathfinding and game decision-making using these search techniques.
- To develop hands-on skills in simulating and visualizing AI search processes in Python, enabling better understanding of algorithm behavior and performance.

Introduction

This lab introduces students to fundamental Artificial Intelligence (AI) search algorithms Breadth-First Search (BFS), Depth-First Search (DFS), A*, and Min-Max and demonstrates their implementation on the Raspberry Pi platform. These algorithms are essential for solving decision-making and path-finding problems in both theoretical and real-world scenarios. From robot navigation and game AI to route optimization and embedded systems, such algorithms form the backbone of intelligent systems. By leveraging the computational power and flexibility of Raspberry Pi, this lab bridges the gap between abstract algorithmic logic and practical, real-time applications. Students will gain hands-on experience coding these algorithms in Python and observing their behavior in simulated or physical environments, laying the foundation for more advanced AI-driven system development.

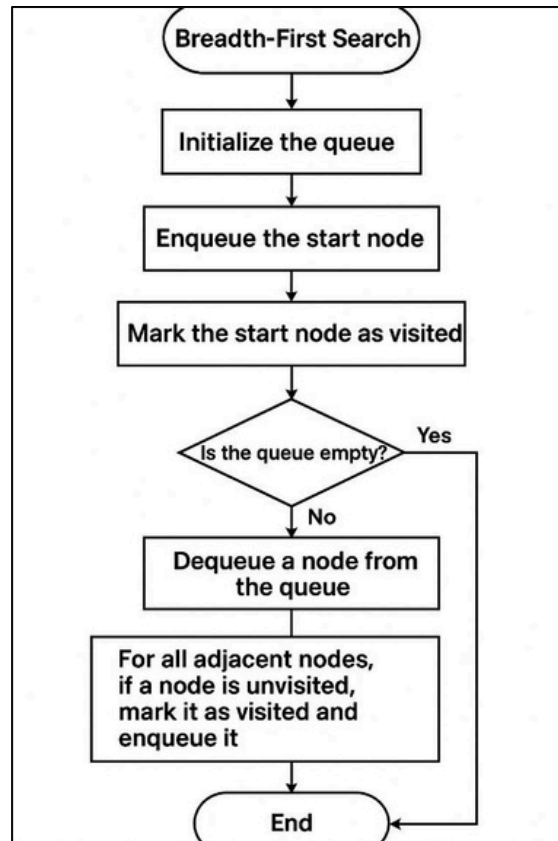
Breath First Search Algorithm

Breadth-First Search (BFS) is an uninformed search algorithm used for traversing or searching tree or graph data structures. It explores all the neighbor nodes at the present depth prior to moving on to the nodes at the next depth level. BFS guarantees the shortest path in an unweighted graph, making it useful for applications like:

- Shortest route in maps
- Web crawlers
- Network broadcasting
- Puzzle solving (e.g., Rubik's Cube, 8-puzzle)

It uses a queue data structure to track the next node to be explored, ensuring a level-by-level traversal.

The BFS can be illustrated by following flowchart



The pseudocode for BFS is given below

- **Initialize:**
 - Create a queue Q.
 - Mark the start node as visited and enqueue it into Q.
- **While** the queue Q is not empty:
 - Dequeue a node from the queue.
 - Process the current node.
 - For each neighbor of the current node:
 - If the neighbor is not visited:
 - Mark it as visited.
 - Enqueue it into Q.
- **End** when the queue is empty, or the goal is found.

Implementation using Python

Import Libraries

```
import matplotlib.pyplot as plt
from collections import deque
```

Define Maze

```
maze = [
    [0,1,0,0,0],
    [0,1,0,1,0],
    [0,0,0,1,0],
    [1,1,0,0,0],
    [0,0,0,1,0]
]

start = (0,0)
goal = (4,4)

rows = len(maze)
cols = len(maze[0])

# Possible movements: Up, Down, Left, Right
directions = [(-1,0),(1,0),(0,-1),(0,1)]
```

BFS Algorithm

```
def bfs(maze, start, goal):

    rows = len(maze)
    cols = len(maze[0])

    visited = [[False for _ in range(cols)] for _ in range(rows)]
    parent = [[None for _ in range(cols)] for _ in range(rows)]

    queue = deque()
    queue.append(start)

    visited[start[0]][start[1]] = True
```

```

while queue:

    x,y=queue.popleft()

    if (x, y) == goal:
        break

    for dx,dy in directions:

        nx = x + dx
        ny = y + dy

        if 0<=nx<rows and 0 <= ny < cols:

            if not visited[nx][ny] and maze[nx][ny] == 0:

                queue.append((nx,ny))
                visited[nx][ny] = True
                parent[nx][ny] = (x,y)

    # Reconstruct path
    path = []
    node = goal

    while node:
        path.append(node)
        node=parent[node[0]][node[1]]

    path.reverse()

    return path, visited

```

Visualization Function

```

def plot_maze(maze, path, visited):
    rows=len(maze)
    cols=len(maze[0])
    plt.figure(figsize=(6,6))
    for i in range(rows):
        for j in range(cols):
            if maze[i][j] == 1:
                plt.scatter(j, i, marker='s', s=500)

            elif visited[i][j]:
                plt.scatter(j, i, marker='o', s=200)

```

```

#Drawpath
px=[p[1] for p in path]
py=[p[0] for p in path]

plt.plot(px, py, linewidth=3)

plt.scatter(start[1], start[0], s=300)
plt.scatter(goal[1], goal[0], s=300)

plt.gca().invert_yaxis()
plt.grid(True)

plt.title("BFS Pathfinding")

plt.show()

```

Run BFS

```

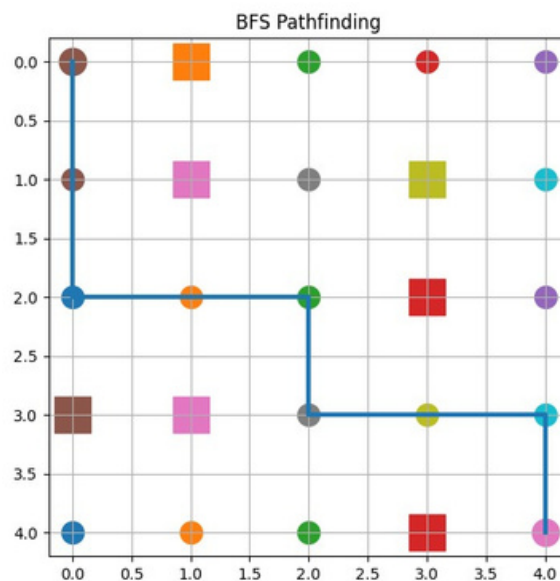
path, visited = bfs(maze, start, goal)

print("Shortest Path using BFS:")
print(path)

plot_maze(maze, path, visited)

```

Output



Depth First Search (DFS)

Depth First Search (DFS) is a classical search algorithm widely used in the field of Artificial Intelligence (AI) for solving problems that require systematic exploration of decision spaces, such as pathfinding, problem solving, and game playing. It belongs to the family of uninformed (or blind) search algorithms, which do not have any prior knowledge about the goal's location in the search space.

In AI, many problems can be represented as a search tree or graph, where each node represents a state, and edges represent transitions or actions. DFS explores this space by starting at the root (initial state) and expanding **the** deepest unexplored node first. This "depth-first" behavior means the algorithm follows one path as far as it can go before backtracking to try alternative paths. The search continues until a goal state is found or all paths are exhausted.

DFS is particularly useful in scenarios where:

- ✓ The solution is deep in the tree, far from the root.
- ✓ Memory resources are limited, since DFS has lower memory requirements compared to Breadth First Search (BFS).
- ✓ Complete traversal of the search space is necessary (e.g., solving puzzles, game moves, or generating all possible solutions).

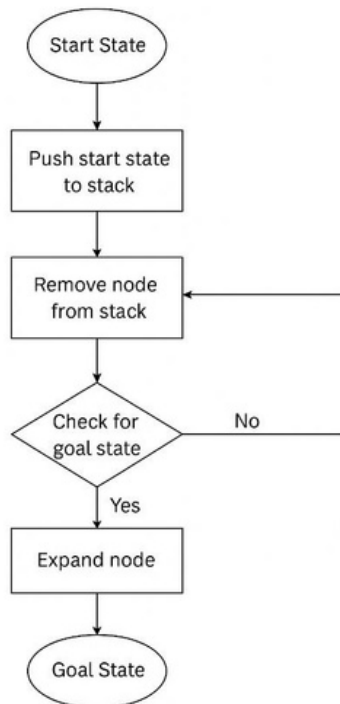
Role in AI Applications:

- ✓ State space exploration in puzzle solvers and logic problems.
- ✓ Planning and scheduling, where sequences of actions are explored.
- ✓ Game AI, for simulating possible moves in depth (e.g., Chess, Tic-Tac-Toe).
- ✓ Backtracking problems, such as N-Queens or solving mazes.
- ✓ Semantic parsing in natural language processing.

Although DFS is not optimal or complete in infinite-depth spaces, its simplicity and depth-oriented behavior make it a powerful tool in various AI domains, especially when combined with techniques like depth limiting **or** iterative deepening to avoid infinite loops and improve efficiency.

DFS can be illustrated by following flow chart.

Depth-First Search



Pseudo code for DFS is given below.

- Initialize an empty stack.
- Push the starting node to the stack and mark it as visited.
- While the stack is not empty:
 - Pop a node from the stack and process it.
 - For each unvisited neighbor of that node:
 - o Push the neighbor to the stack.
 - o Mark it as visited.

Implementation on Python

```
import matplotlib.pyplot as plt
import numpy as np

# Create a maze (0 = path, 1 = wall)
maze = [
    [0, 1, 1, 1, 1],
    [0, 0, 1, 0, 0],
    [1, 0, 0, 0, 1],
    [1, 1, 1, 0, 1],
    [1, 1, 1, 0, 0]
]

start = (0, 0)
end = (4, 4)

def dfs(maze, start, end):
    stack = [(start, [start])]
    visited = set()

    while stack:
        (current, path) = stack.pop()
        if current == end:
            return path
        visited.add(current)

        x, y = current
        directions = [(0,1),(1,0),(0,-1),(-1,0)]

        for dx, dy in directions:
            nx, ny = x + dx, y + dy
            if (0 <= nx < len(maze) and 0 <= ny < len(maze[0])
                and maze[nx][ny] == 0 and (nx, ny) not in visited):
                stack.append(((nx, ny), path + [(nx, ny)]))

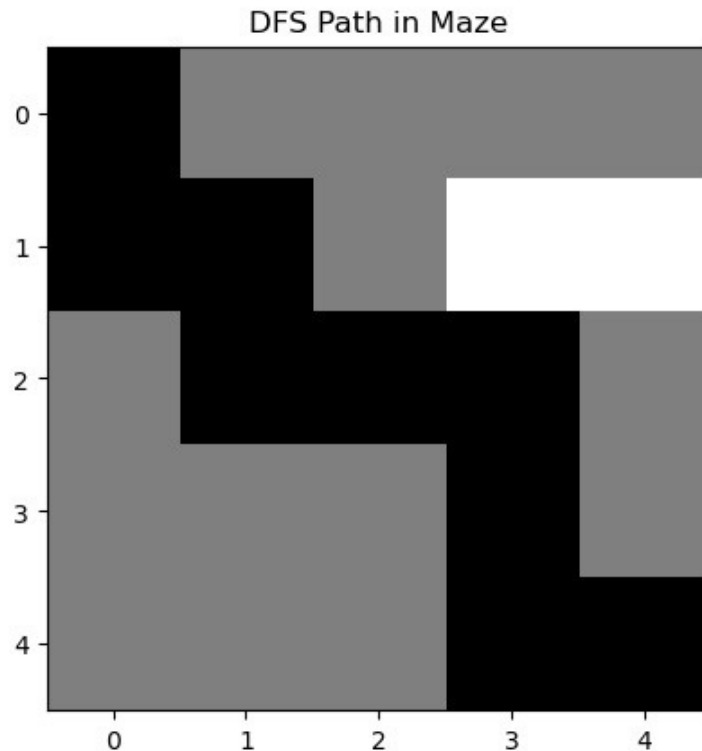
    return None

path = dfs(maze, start, end)
print("DFS Path:", path)

# Visualize
def draw_maze_with_path(maze, path):
    maze_arr = np.array(maze)
    for (x, y) in path:
        maze_arr[x][y] = 2 # Mark path
    plt.imshow(maze_arr, cmap="gray_r")
    plt.title("DFS Path in Maze")
    plt.show()
```

```
if path:
    draw_maze_with_path(maze, path)
else:
    print("No path found.")
```

Output



A* (A-Star) Algorithm

The A* algorithm is a powerful and widely used informed search algorithm in Artificial Intelligence, particularly in pathfinding and graph traversal. It combines the strengths of Dijkstra's algorithm and Greedy Best-First Search by considering both the cost to reach the current node and the estimated cost to reach the goal.

A* uses a heuristic function ($h(n)$) along with the actual cost ($g(n)$) to evaluate nodes using the function:

$$f(n)=g(n)+h(n)$$

Where:

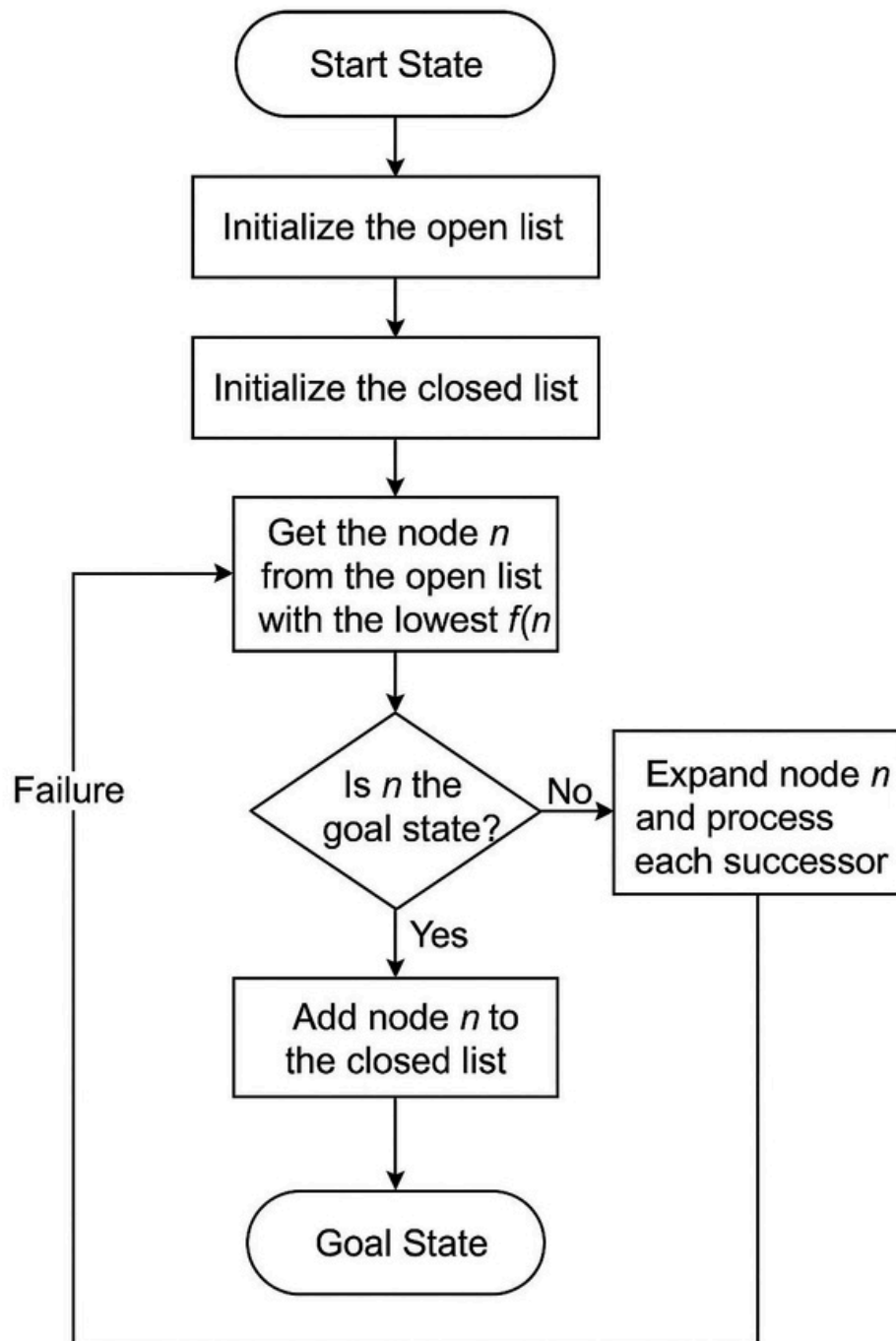
- $g(n)$ = the actual cost from the start node to node n
- $h(n)$ = the heuristic estimated cost from node n to the goal
- $f(n)$ = the estimated total cost of the path through node n

This makes A* complete, optimal (if the heuristic is admissible), and efficient for many practical use cases in AI like robotics, gaming, and route planning.

Pseudo Code for A-Star Algorithm

- Initialize the open list with the start node.
- Initialize the closed list as empty.
- Repeat:
 - Get the node n from the open list with the lowest $f(n)$.
 - If n is the goal node, return the path.
 - Move n from open list to closed list.
 - For each neighbor m of n :
 - If m is in the closed list, skip it.
 - Calculate $g(m)$, $h(m)$, and $f(m)$.
 - If m is not in the open list, add it.
 - If m is already in the open list with a higher cost, update it.
- If the open list is empty, return failure.

Flowchart for A-Star Algorithm is shown below



Implementation of A-Star Algorithm in Python

```
import heapq
import matplotlib.pyplot as plt
import numpy as np

# Maze: 0 = path, 1 = wall
maze = [
    [0, 1, 0, 0, 0],
    [0, 1, 0, 1, 0],
    [0, 0, 0, 1, 0],
    [1, 1, 0, 1, 0],
    [0, 0, 0, 0, 0]
]

start = (0, 0)
end = (4, 4)

# Heuristic function (Manhattan distance)
def heuristic(a, b):
    return abs(a[0] - b[0]) + abs(a[1] - b[1])

def astar(maze, start, end):
    rows, cols = len(maze), len(maze[0])
    open_set = []
    heapq.heappush(open_set, (0 + heuristic(start, end), 0, start))
    came_from = {}

    g_score = {start: 0}

    while open_set:
        _, cost, current = heapq.heappop(open_set)

        if current == end:
            path = []
            while current in came_from:
                path.append(current)
                current = came_from[current]
            path.append(start)
            path.reverse()
            return path

        neighbors = [(0, 1), (1, 0), (-1, 0), (0, -1)]
        for dx, dy in neighbors:
            neighbor = (current[0] + dx, current[1] + dy)
            x, y = neighbor
```

```

        if 0 <= x < rows and 0 <= y < cols and maze[x][y] == 0:
            tentative_g_score = g_score[current] + 1
            if neighbor not in g_score or tentative_g_score <
g_score[neighbor]:
                g_score[neighbor] = tentative_g_score

            f_score = tentative_g_score + heuristic(neighbor, end)
            heapq.heappush(open_set, (f_score, tentative_g_score,
neighbor))

            came_from[neighbor] = current

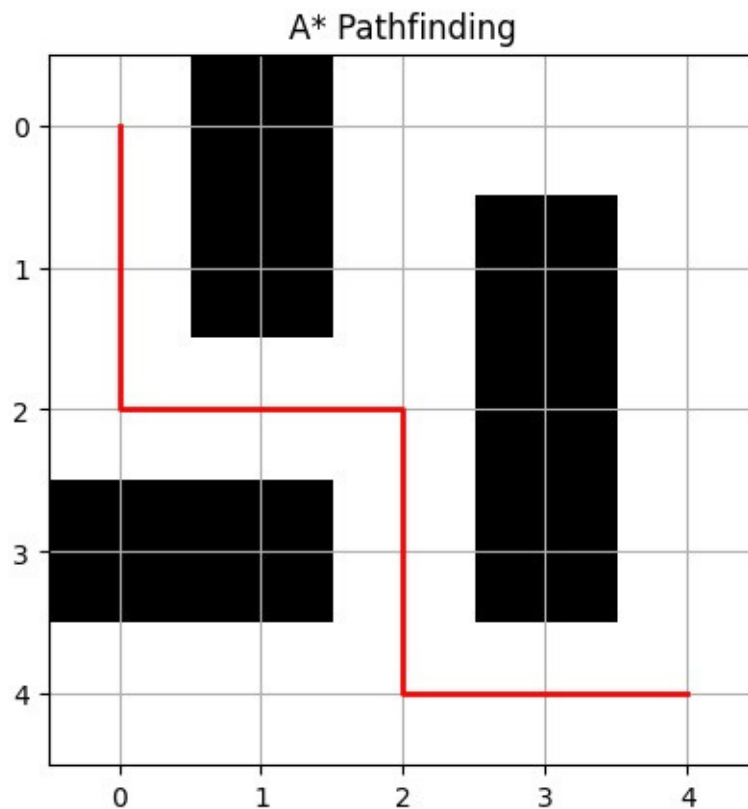
    return None

# Visualization
def draw_maze_with_path(maze, path):
    maze_array = np.array(maze)
    plt.imshow(maze_array, cmap='gray_r')
    if path:
        x_coords = [y for x, y in path]
        y_coords = [x for x, y in path]
        plt.plot(x_coords, y_coords, color='red', linewidth=2)
    plt.title("A* Pathfinding")
    plt.xticks(np.arange(len(maze[0])))
    plt.yticks(np.arange(len(maze)))
    plt.grid(True)
    plt.show()

# Run A* and draw result
path = astar(maze, start, end)
print("Shortest Path:", path)
draw_maze_with_path(maze, path)

```

Output



Shortest Path: [(0, 0), (1, 0), (2, 0), (2, 1), (2, 2), (3, 2), (4, 2), (4, 3), (4, 4)]

Mini Max Algorithm

The Minimax algorithm is a decision-making algorithm used in two-player turn-based games like chess, tic-tac-toe, and checkers. It is a recursive strategy used to choose the optimal move for a player, assuming that the opponent also plays optimally. The algorithm explores all possible moves (a decision tree), simulates all potential responses by the opponent, and calculates a value for each final state of the game. The player then selects the move that maximizes their minimum gain, hence the name *Minimax*.

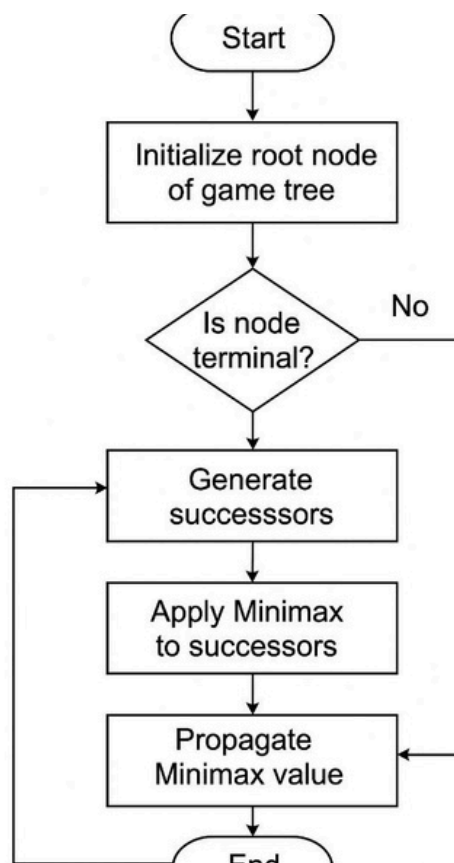
In Artificial Intelligence, Minimax forms the backbone of intelligent game agents. It's not only about winning it's about thinking ahead, simulating the opponent's decisions, and planning accordingly. This makes it an essential algorithm in the field of adversarial search problems. Important applications are

- ✓ **Core of Game AI:** Used in board games, strategy games, and decision-making in adversarial environments.
- ✓ **Foundation for Alpha-Beta Pruning:** A more efficient variant of Minimax.
- ✓ **Teaches Lookahead and Tree Search:** Forms a strong base for decision trees and adversarial planning.
- ✓ **AI that “Thinks”:** Emulates human reasoning in competitive settings.

Comparison with Other Search Algorithms

Algorithm	Type	Use Case	Minimax Difference
BFS/DFS	Tree/Graph	Pathfinding, Maze-solving	Doesn't assume an opponent
A*	Heuristic	Shortest-path in weighted graphs	Doesn't deal with turn-based adversaries
Minimax	Adversarial	Games, strategy, decision planning	Specifically handles 2-player games

Flow chart of Min Max algorithm is given below



Min Max algorithm can be coded using following pseudo code

```
1. function minimax(node, depth, maximizingPlayer):
2.   if node is a terminal node or depth == 0:
3.     return evaluate(node)
4.   if maximizingPlayer:
5.     maxEval = -infinity
6.     for each child of node:
7.       eval = minimax(child, depth - 1, false)
8.       maxEval = max(maxEval, eval)
9.     return maxEval
10.  else:
11.    minEval = +infinity
12.    for each child of node:
13.      eval = minimax(child, depth - 1, true)
14.      minEval = min(minEval, eval)
15.    return minEval
```

Implementation using Python

Import Libraries

```
import math
```

Create the Tic-Tac-Toe Board

```
board = [
    ["X", "O", "X"],
    ["_", "O", "_"],
    ["_", "_", "X"]
]

def print_board(board):
    for row in board:
        print(" ".join(row))
    print()

print("Current Board:")
print_board(board)
```

Check Winner Function

```
def check_winner(board):  
  
    # Check rows  
    for row in board:  
        if row[0] == row[1] == row[2] and row[0] != "_":  
            return row[0]  
  
    # Check columns  
    for col in range(3):  
        if board[0][col] == board[1][col] == board[2][col] and board[0][col] != "_":  
            return board[0][col]  
  
    # Check diagonals  
    if board[0][0] == board[1][1] == board[2][2] and board[0][0] != "_":  
        return board[0][0]  
  
    if board[0][2] == board[1][1] == board[2][0] and board[0][2] != "_":  
        return board[0][2]  
  
    return None
```

Get Empty Positions

```
def get_empty_cells(board):  
  
    cells = []  
  
    for i in range(3):  
        for j in range(3):  
            if board[i][j] == "_":  
                cells.append((i,j))  
  
    return cells
```

MiniMax Algorithm

```
# Minimax Algorithm  
# The Minimax algorithm is a decision-making algorithm used in Artificial Intelligence  
# for two-player games such as Tic-Tac-Toe. It assumes that both players play  
# optimally.  
# The algorithm recursively explores all possible moves and assigns scores to game  
# states:  
# +1 → AI player (O) wins
```

```

def minimax(board, is_maximizing):

    winner = check_winner(board)

    if winner == "O":
        return 1

    if winner == "X":
        return -1

    if not get_empty_cells(board):
        return 0

    if is_maximizing:

        best_score = -math.inf

        for (i, j) in get_empty_cells(board):

            board[i][j] = "O"

            score = minimax(board, False)

            board[i][j] = "_"

            best_score = max(score, best_score)

        return best_score

    else:

        best_score = math.inf

        for (i, j) in get_empty_cells(board):

            board[i][j] = "X"

            score = minimax(board, True)

            board[i][j] = "_"

            best_score = min(score, best_score)

        return best_score

```

Find Best Move for AI

```
def best_move(board):  
  
    best_score = -math.inf  
    move = None  
  
    for (i,j) in get_empty_cells(board):  
  
        board[i][j] = "O"  
  
        score = minimax(board, False)  
  
        board[i][j] = "_"  
  
        if score > best_score:  
            best_score = score  
            move = (i,j)  
  
    return move
```

Execute Best Move

```
move = best_move(board)  
  
print("Best Move for AI (O):", move)  
  
board[move[0]][move[1]] = "O"  
  
print("\nBoard After AI Move:")  
print_board(board)
```

Output

```
Best Move for AI (O): (1, 2)
```

```
Board After AI Move:
```

```
X O X  
_ O O  
_ _ X
```

LAB TASKS

1. Design an 8×8 grid maze in Python where 0 represents an open path and 1 represents walls. Simulate a robot finding the shortest path from the top-left corner to the bottom-right corner using the Breadth-First Search (BFS) algorithm. Visually highlight the explored cells and the final shortest path using a graphical plot.
2. Implement a Depth-First Search (DFS) based exploration algorithm to locate a hidden “treasure” inside a grid environment. Simulate the exploration process and visually mark the visited cells during traversal. The algorithm should explore as deep as possible before backtracking.
3. Simulate a smart delivery bot in a grid-based map where certain cells represent blocked roads. Implement the A* search algorithm with the Manhattan distance heuristic to find the optimal path from location A (start) to location B (destination). Display the search process and final path visually.
4. Develop a Tic-Tac-Toe game in Python where a human player competes against an AI agent. The AI should use the Minimax algorithm to select optimal moves and avoid losing. Display the game board and results visually using colored output or graphical representation.
5. Implement an interactive grid-based pathfinding simulation using the A* algorithm in Python. Visualize the robot’s movement step-by-step across the grid as it navigates from the start cell to the goal cell. Each step of the computed path should be highlighted sequentially to simulate movement.

1. Design an 8×8 grid maze in Python where 0 represents an open path and 1 represents walls. Simulate a robot finding the shortest path from the top-left corner to the bottom-right corner using the Breadth-First Search (BFS) algorithm. Visually highlight the explored cells and the final shortest path using a graphical plot.

Task 1: BFS Robot Path Finding in 8×8 Maze

```

from collections import deque
import matplotlib.pyplot as plt
import numpy as np

maze = [
    [0,0,0,1,0,0,0,0],
    [1,1,0,1,0,1,1,0],
    [0,0,0,0,0,0,1,0],
    [0,1,1,1,1,0,1,0],
    [0,0,0,0,1,0,0,0],
    [1,1,1,0,1,1,1,0],
    [0,0,0,0,0,0,0,0],
    [0,1,1,1,1,1,1,0]
]

start = (0,0)
goal = (7,7)

def bfs(maze,start,goal):

    rows=len(maze)
    cols=len(maze[0])

    queue=deque([start])
    visited=set([start])
    parent={}

    directions=[(0,1),(1,0),(0,-1),(-1,0)]

    while queue:
        current=queue.popleft()

        if current==goal:
            break

        x,y=current

        for dx,dy in directions:
            nx,ny=x+dx,y+dy

            if 0<=nx<rows and 0<=ny<cols:
                if maze[nx][ny]==0 and (nx,ny) not in visited:
                    queue.append((nx,ny))
                    visited.add((nx,ny))
                    parent[(nx,ny)] = current

    path=[]
    node=goal

    while node in parent:
        path.append(node)
        node=parent[node]

    path.append(start)
    path.reverse()

    return path,visited

path,visited=bfs(maze,start,goal)

maze_arr=np.array(maze)

plt.imshow(maze_arr,cmap="gray_r")

for x,y in visited:
    plt.scatter(y,x,s=100)

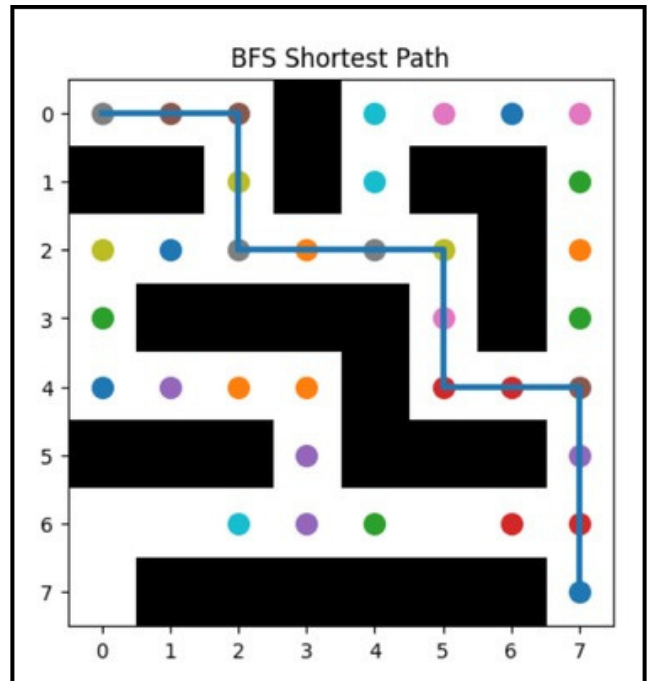
px=[p[1] for p in path]
py=[p[0] for p in path]

plt.plot(px,py,linewidth=3)

plt.title("BFS Shortest Path")
plt.show()

print("Shortest Path:")
print(path)

```



Shortest Path:
 [(0, 0), (0, 1), (0, 2), (1, 2), (2, 2),
 (2, 3), (2, 4), (2, 5), (3, 5), (4, 5),
 (4, 6), (4, 7), (5, 7), (6, 7), (7, 7)]

2. Implement a Depth-First Search (DFS) based exploration algorithm to locate a hidden “treasure” inside a grid environment. Simulate the exploration process and visually mark the visited cells during traversal. The algorithm should explore as deep as possible before backtracking.

Task 2: DFS Treasure Hunt

```
import matplotlib.pyplot as plt
import numpy as np

grid = [
    [0,0,0,1,0],
    [1,1,0,1,0],
    [0,0,0,0,0],
    [0,1,1,1,0],
    [0,0,0,0,'T']
]

start=(0,0)

visited=[]

def dfs(grid,current):

    x,y=current

    if grid[x][y]=='T':
        return True

    visited.append(current)

    directions=[(0,1),(1,0),(0,-1),(-1,0)]

    for dx,dy in directions:

        nx=x+dx
        ny=y+dy

        if 0<=nx<len(grid) and 0<=ny<len(grid[0]):

            if grid[nx][ny]!=1 and (nx,ny) not in visited:

                if dfs(grid,(nx,ny)):
                    return True

    return False

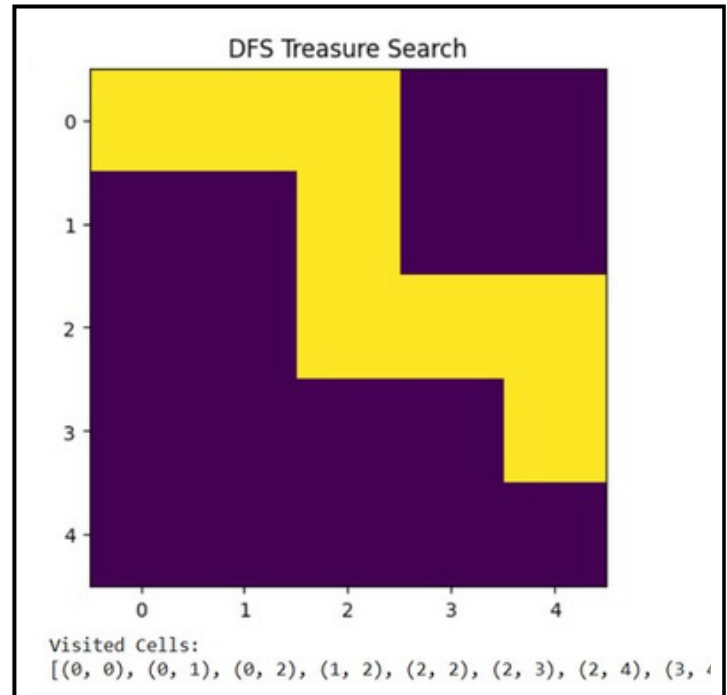
dfs(grid,start)

maze=np.zeros((5,5))

for x,y in visited:
    maze[x][y]=2

plt.imshow(maze,cmap="viridis")
plt.title("DFS Treasure Search")
plt.show()

print("Visited Cells:")
print(visited)
```



3. Simulate a smart delivery bot in a grid-based map where certain cells represent blocked roads. Implement the A* search algorithm with the Manhattan distance heuristic to find the optimal path from location A (start) to location B (destination). Display the search process and final path visually.

Task 3: Smart Delivery Bot using A*

```

import heapq
import matplotlib.pyplot as plt
import numpy as np

grid = [
    [0,0,0,0,0],
    [1,1,0,1,0],
    [0,0,0,1,0],
    [0,1,0,0,0],
    [0,0,0,1,0]
]

start=(0,0)
goal=(4,4)

def heuristic(a,b):
    return abs(a[0]-b[0])+abs(a[1]-b[1])

def astar(grid,start,goal):

    open_list=[]
    heapq.heappush(open_list,(0,start))

    parent={}
    g={start:0}

    while open_list:
        _,current=heapq.heappop(open_list)

        if current==goal:
            path=[]

            while current in parent:
                path.append(current)
                current=parent[current]

            path.append(start)
            path.reverse()
            return path

        x,y=current

        for dx,dy in [(0,1),(1,0),(0,-1),(-1,0)]:
            nx,ny=x+dx,y+dy

            if 0<=nx<5 and 0<=ny<5 and grid[nx][ny]==0:
                new_cost=g[current]+1

                if (nx,ny) not in g or new_cost<g[(nx,ny)]:
                    g[(nx,ny)] = new_cost

```

```

        f=new_cost+heuristic((nx,ny),goal)
        heapq.heappush(open_list,(f,(nx,ny)))

        parent[(nx,ny)] = current

    path=astar(grid,start,goal)

    arr=np.array(grid)

    plt.imshow(arr,cmap="gray_r")

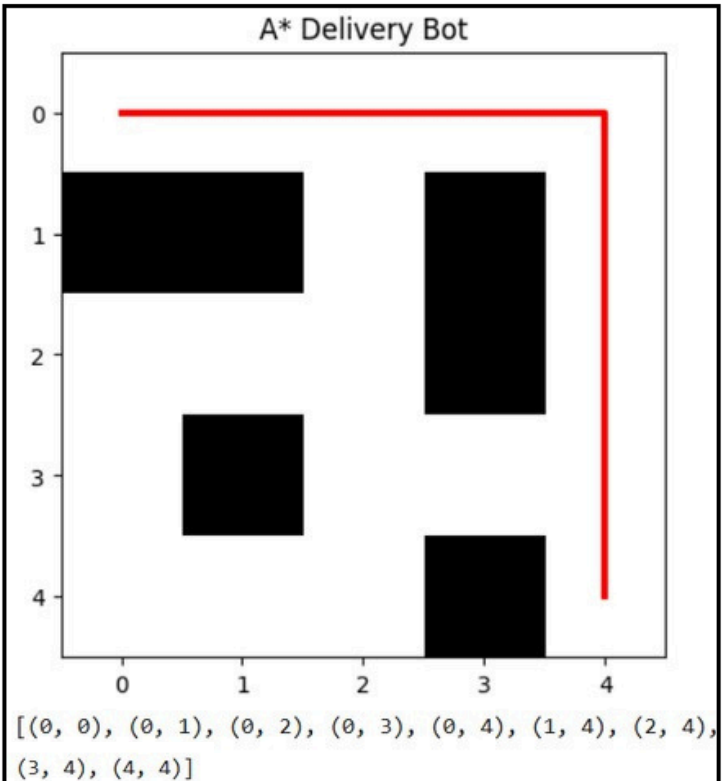
    x=[p[1] for p in path]
    y=[p[0] for p in path]

    plt.plot(x,y,'r',linewidth=3)

    plt.title("A* Delivery Bot")
    plt.show()

    print(path)

```



4. Develop a Tic-Tac-Toe game in Python where a human player competes against an AI agent. The AI should use the Minimax algorithm to select optimal moves and avoid losing. Display the game board and results visually using colored output or graphical representation.

Task 4: Tic Tac Toe using Minimax

```

import math

board = [
    ["_", "_", "_"],
    ["_", "_", "_"],
    ["_", "_", "_"]
]

def print_board():
    print()
    for row in board:
        print(" ".join(row))
    print()

def winner():
    for row in board:
        if row[0] == row[1] == row[2] and row[0] != "_":
            return row[0]

    for col in range(3):
        if board[0][col] == board[1][col] == board[2][col] and board[0][col] != "_":
            return board[0][col]

    if board[0][0] == board[1][1] == board[2][2] and board[0][0] != "_":
        return board[0][0]

    if board[0][2] == board[1][1] == board[2][0] and board[0][2] != "_":
        return board[0][2]

    return None

def empty_cells():
    cells = []
    for i in range(3):
        for j in range(3):
            if board[i][j] == "_":
                cells.append((i,j))
    return cells

def minimax(is_max):
    w = winner()

    if w == "O":
        return 1

    if w == "X":
        return -1

    if not empty_cells():
        return 0

    if is_max:
        best = -math.inf

        for i,j in empty_cells():
            board[i][j] = "O"

            score = minimax(False)

            board[i][j] = "_"

            best = max(best, score)

        return best
    else:
        best = math.inf

        for i,j in empty_cells():
            board[i][j] = "X"

            score = minimax(True)

            board[i][j] = "_"

            best = min(best, score)

        return best

```

```

        best = min(best, score)

    return best

def ai_move():
    best_score = -math.inf
    move = None

    for i,j in empty_cells():
        board[i][j] = "O"

        score = minimax(False)

        board[i][j] = "_"

        if score > best_score:
            best_score = score
            move = (i,j)

    board[move[0]][move[1]] = "O"

print("TIC TAC TOE")
print("You = X")
print("AI = O")

while True:
    print_board()

    row = int(input("Enter row (0-2): "))
    col = int(input("Enter column (0-2): "))

    if board[row][col] != "_":
        print("Cell already occupied!")
        continue

    board[row][col] = "X"

    if winner() == "X":
        print_board()
        print("You Win!")
        break

    if not empty_cells():
        print_board()
        print("Draw!")
        break

    ai_move()

    if winner() == "O":
        print_board()
        print("AI Wins!")
        break

    if not empty_cells():
        print_board()
        print("Draw!")
        break

```

Output:

```
TIC TAC TOE
You = X
... AI = O

_ _ _
_ _ _
_ _ _

Enter row (0-2): 1
Enter column (0-2): 1

O _ _
_ X _
_ _ _

Enter row (0-2): 0
Enter column (0-2): 1

O X _
_ X _
_ O _

Enter row (0-2): 0
Enter column (0-2): 2

O X X
_ X _
O O _

Enter row (0-2): 2
Enter column (0-2): 2

O X X
O X _
O O X

AI Wins!
```

5. Implement an interactive grid-based pathfinding simulation using the A* algorithm in Python. Visualize the robot's movement step-by-step across the grid as it navigates from the start cell to the goal cell. Each step of the computed path should be highlighted sequentially to simulate movement.

Task 5: Step-by-Step Robot Movement using A*

```
import matplotlib.pyplot as plt
import numpy as np
import time

path = [
    (0,0),
    (1,0),
    (2,0),
    (2,1),
    (2,2),
    (3,2),
    (4,2),
    (4,3),
    (4,4)
]

grid=np.zeros((5,5))

for step in path:

    plt.clf()

    temp=grid.copy()

    x,y=step

    temp[x][y]=5

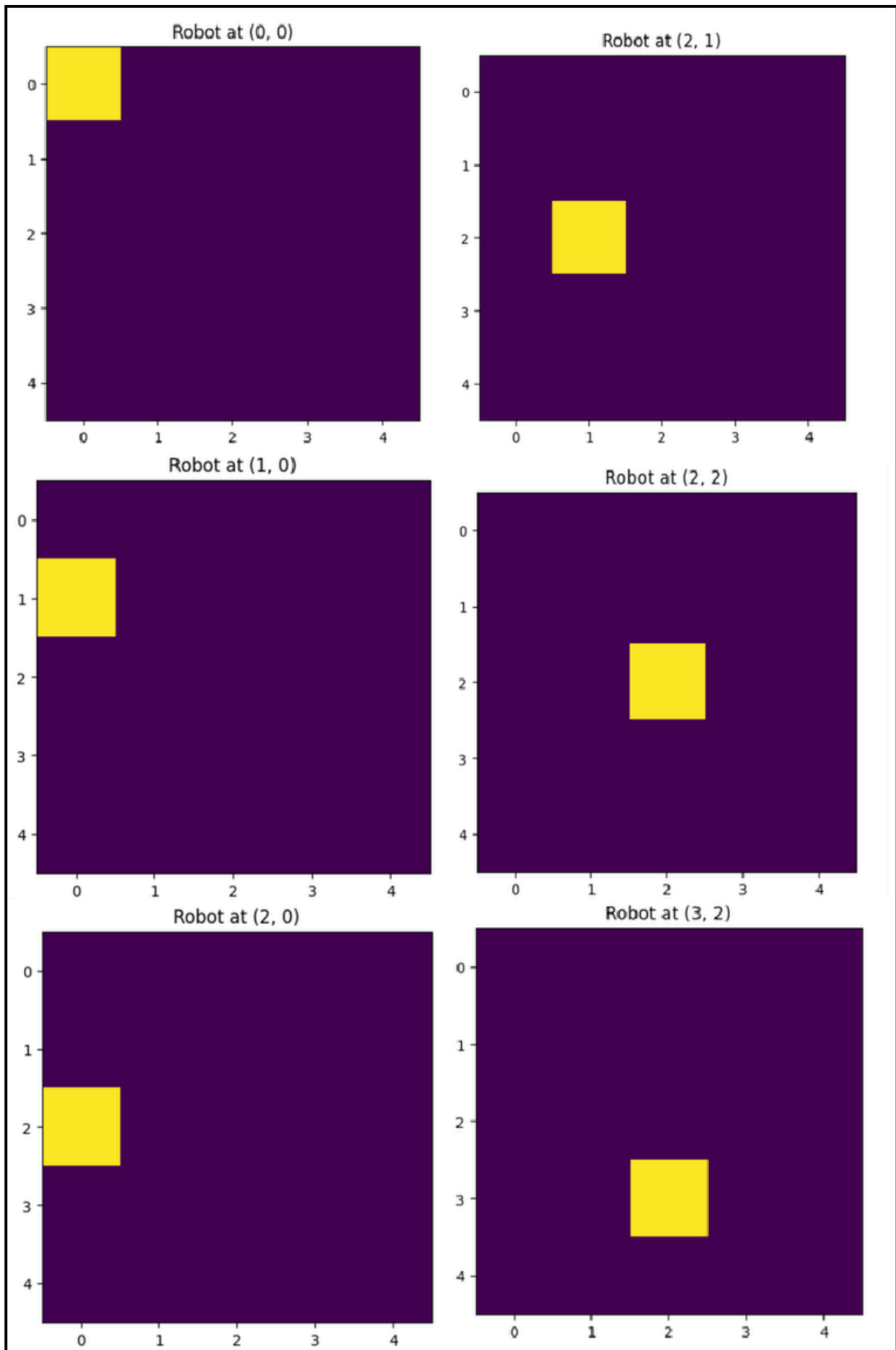
    plt.imshow(temp,cmap="viridis")

    plt.title(f"Robot at {step}")

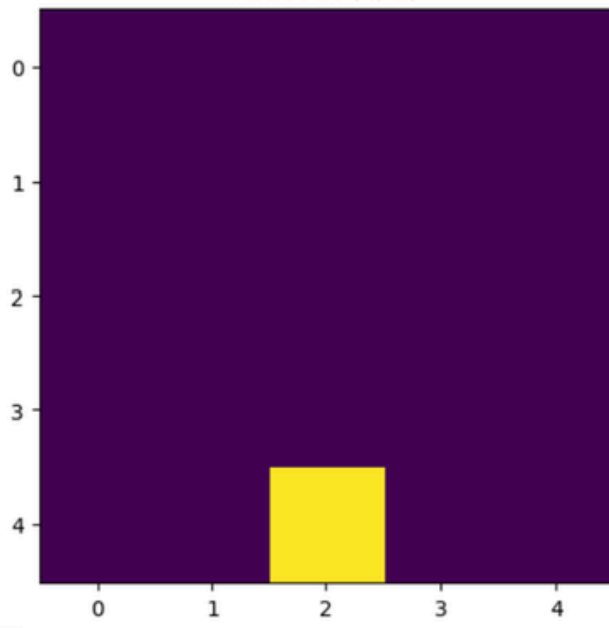
    plt.pause(0.5)

plt.show()
```

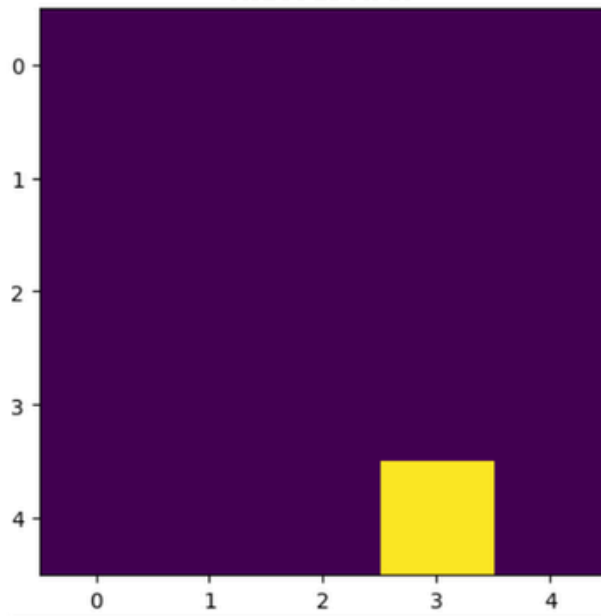
Output:



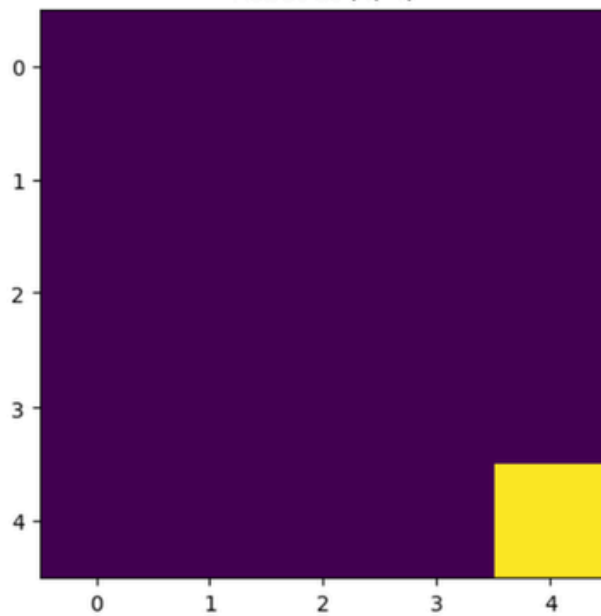
Robot at (4, 2)



Robot at (4, 3)



Robot at (4, 4)



International Islamic University Islamabad
Faculty of Engineering and Technology
Department of Electrical Engineering

Artificial Intelligence & Machine Learning LAB

Lab No. 6

Design and Implementation of AI Search Algorithms for Pathfinding and Game Decision Making

Name: Bushra Nazir Reg. No.: 1071-F22

S. No.	Criteria	Allocated Percentage	Level 1 (0%)	Level 2 (25%)	Level 3 (50%)	Level 4 (75%)	Level 5 (100%)	Marks Obtained
(A) <u>PSYCHOMOTOR</u> (To be judged in the lab during experiment)								
1	Practical Implementation OR Connectivity/Arrangement of Equipment	5	Absent	Degree of errors in applying procedural knowledge or Degree of errors in arrangement of equipment:				
			0	With several critical errors, Incomplete and Not Neat	With few errors, Incomplete and not Neat	With some errors, Complete but not Neat	Without errors, Complete and Neat	
				1.25	2.5	3.75	5	
2	Use of Equipment OR Use of Simulation/Programming Tool	2	Absent	Use of tools, equipment and materials with:				
			0	Limited Competence	Some Competence	Considerable Competence	Good Competence	
				0.5	1	1.5	2	
3	Safety (Optional)	0	Absent	Degree of reminders to follow safety procedure with:				
			0	Rarely Follow Safety Rules	Needs Reminders	Needs minor Reminders	Follows Safety Rules	
				0	0	0	0	
SUB-TOTAL (PT)		7	SUB-TOTAL MARKS OBTAINED (PO)					
(B) <u>COGNITIVE</u> (To be judged on the copy of experiment submitted)								
4	Data Record, Analysis and Evaluation OR Algorithm Design	1	Absent	Data record, Analysis and Evaluation or Designed Algorithm is:				
			0	Incorrect /Incomplete	Complete with some errors	Complete with few errors	Complete and Accurate	
				0.25	0.5	0.75	1	
5	Procedural Awareness (Optional)	0	Absent	Procedural awareness of the completed task is:				
			0	Inappropriate	Appropriate with some errors	Appropriate with few errors	Appropriate	
				0	0	0	0	
SUB-TOTAL (CT)		1	SUB-TOTAL MARKS OBTAINED (CO)					
(C) <u>AFFECTIVE</u> (To be judged in the lab during experiment)								
6	Level of Participation & Attitude to Achieve Individual/Group Goals	2	Absent	Degree of positive interaction as an individual or within a group:				
			0	Rare interaction	Some interaction	Acceptable interaction	Good interaction	
				0.5	1	1.5	2	
SUB-TOTAL (AT)		2	SUB-TOTAL MARKS OBTAINED (AO)					

Total Marks (PT + CT + AT) :

10

Instructor

Marks Obtained (PO + CO + AO) :